2026-04-27-jackett-management-ui-and-system-db-design

Jackett Management UI & System Database — Design Spec

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

Field	Value
Date	2026-04-27
Author	Brainstorm session (milos85vasic.2nd@gmail.com + Claude)
Status	Awaiting user review
Phase	D (Go-canonical Jackett surface + SQLite); Phase 2 = port merge search to Go
Supersedes	n/a (extends `2026-04-26-jackett-autoconfig-clean-rebuild-design.md`)

---

1. Problem Statement

The Jackett auto-configuration shipped in commits `45591c2` → `fc1f009` (Layers 1-7) configures private-tracker indexers from `.env` credentials at startup. Verified live behavior on 2026-04-27:

- 4 credential triples discovered (`IPTORRENTS`, `KINOZAL`, `NNMCLUB`, `RUTRACKER`).
- 2 indexers configured (`kinozal`, `rutracker`).
- 1 skipped — `NNMCLUB` (no Jackett indexer in 620-entry catalog; native plugin handles it).

- 1 errored — `IPTORRENTS` (Jackett's `iptorrents` indexer requires a `cookie`, env only had username/password).

Three end-user gaps remain:

1. **No in-dashboard visibility** of which Jackett indexers are configured, their status, or the autoconfig run history. Users must visit Jackett's UI on `:9117` for any introspection or change.
2. **No in-dashboard management** — adding, editing, removing indexers, browsing the 620-indexer catalog, or fixing credential issues (like the `IPTORRENTS_COOKIES` gap) all require leaving Боба.
3. **No persistent canonical store** for credentials, autoconfig history, or catalog metadata. State today is split across `.env` (plaintext, single-process), per-indexer JSON files in `config/jackett/`, and in-memory results.

2. Goals

- A Боба-native UI for full Jackett management (browse 620-indexer catalog, add/edit/remove, test, browse autoconfig history).
- A SQLite system database (`config/boba.db`) as the persistent canonical store for credentials (encrypted), indexer state, catalog cache, autoconfig run history, and indexer-name overrides.
- Credentials encrypted at rest with AES-256-GCM using `BOBA_MASTER_KEY` (auto-generated into `.env` on first boot if missing).
- Bidirectional sync: `.env` → DB on first run; UI changes write to BOTH DB (encrypted) AND `.env` (plaintext mirror) AND Jackett (live config).
- Strong leak-prevention tests asserting no credential value ever appears in logs or HTTP responses.
- Begin Go-canonical migration with a self-contained subsystem (Jackett UI), without disrupting the Python merge service that serves search.
- Zero loose ends at done-time: explicit Definition-of-Done checklist (§10) drives the implementation plan.

3. Non-Goals (explicit Phase 2+)

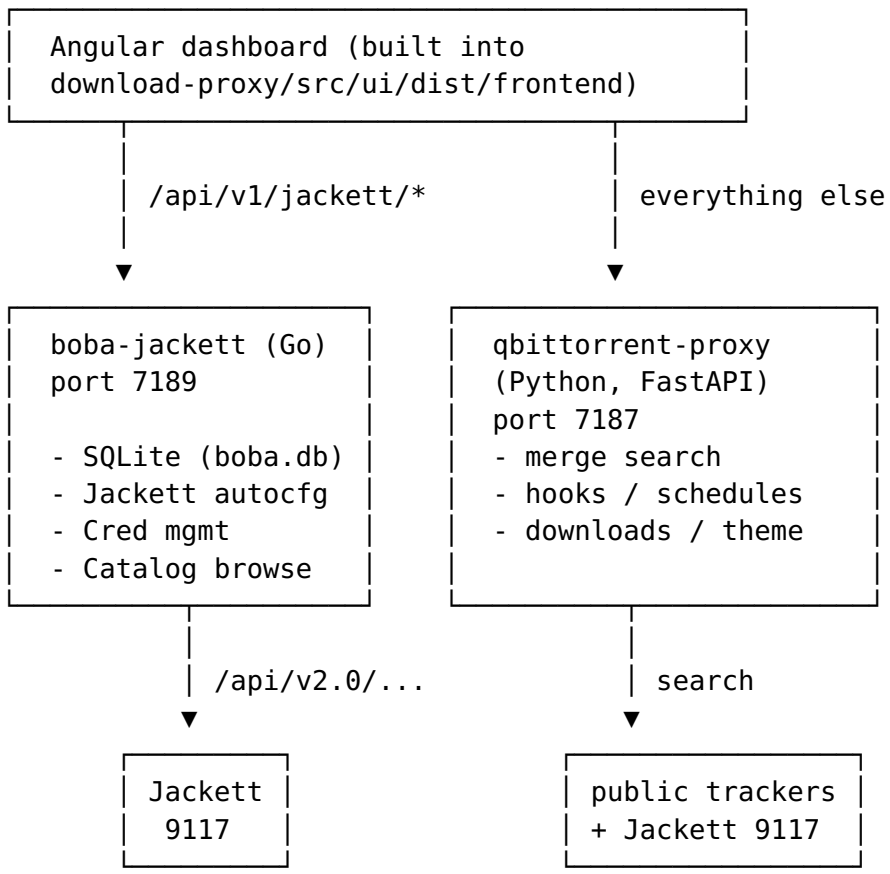
- Porting the merge search path (`download-proxy/src/merge_service/search.py` and friends) to Go. Stays Python until Phase 2.
- Porting hooks / schedules / theme / downloads endpoints to Go. Stays Python.

- Replacing nova3 search plugins (`plugins/*.py`) with Go equivalents. They run inside qBittorrent and must remain Python.
- A new login system for the dashboard. Re-uses existing `admin/admin` session auth.
- Postgres/MySQL support. SQLite only.
- Multi-user authorization. Single-tenant home-server use.

4. Architecture

4.1 Process layout

Two backend processes during Phase D, both behind the existing Angular frontend:



The Angular app calls each backend on its own port; no reverse proxy is added. The existing Python `/api/v1/jackett/autoconfig/last` endpoint is removed; the Go service owns the entire `/api/v1/jackett/*` namespace.

4.2 Container layout

`docker-compose.yml` gains one service:

```yaml
boba-jackett:
  build:
    context: ./qBitTorrent-go
    dockerfile: Dockerfile.jackett        # new, separate from
        existing Dockerfile
  container_name: boba-jackett
  environment:
    - JACKETT_URL=http://localhost:9117
    - JACKETT_API_KEY=${JACKETT_API_KEY:-}
    - BOBA_MASTER_KEY=${BOBA_MASTER_KEY:-}
    - BOBA_DB_PATH=/config/boba.db
    - BOBA_ENV_PATH=/host-env/.env
    - PORT=7189
    - LOG_LEVEL=INFO
  volumes:
    - ./config:/config                    # boba.db lives here
    - ./config/jackett:/jackett-config:ro
    - ./.env:/host-env/.env               # bind-mount for atomic
        updates
  network_mode: host
  restart: unless-stopped
  mem_limit: 256m
  pids_limit: 256
  oom_score_adj: 500
  depends_on:
    jackett:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:7189/healthz
        || exit 1"]
    interval: 30s
    timeout: 5s
    retries: 5
    start_period: 30s
```

`start.sh` extends `extract_jackett_key` with a similar `ensure_boba_master_key` step that runs *before* `start_container`, generating + writing `BOBA_MASTER_KEY` to `.env` if missing. The Go binary itself ALSO performs this generation at startup (defense in depth and also for cases where the host script is bypassed).

4.3 Go module layout

```
qBitTorrent-go/
├── cmd/
│   ├── qbittorrent-proxy/main.go      # existing
│   ├── webui-bridge/main.go           # existing
│   └── boba-jackett/main.go           # NEW
├── internal/
│   ├── api/                           # existing handlers stay
│   ├── client/                        # existing
│   ├── jackett/                       # NEW — autoconfig port +
```

```
catalog client
│   │   ├── autoconfig.go
│   │   ├── catalog.go
│   │   ├── client.go
│   │   ├── matcher.go
│   │   └── *_test.go
│   ├── db/                              # NEW — SQLite + migrations
+ crypto
│   │   ├── conn.go
│   │   ├── crypto.go                    # AES-256-GCM helpers
│   │   ├── migrate.go                   # embedded migrations
stepper
│   │   ├── migrations/
│   │   │   └── 001_init.sql
│   │   └── repos/                       # one repo per table
│   │       ├── credentials.go
│   │       ├── indexers.go
│   │       ├── catalog.go
│   │       ├── runs.go
│   │       └── overrides.go
│   ├── envfile/                         # NEW — atomic .env reader
+ writer
│   │   ├── parse.go
│   │   ├── write.go                     # tmp + fsync + rename
│   │   └── *_test.go
│   ├── jackettapi/                      # NEW — HTTP handlers for /
api/v1/jackett/*
│   │   ├── credentials.go
│   │   ├── indexers.go
│   │   ├── catalog.go
│   │   ├── runs.go
│   │   ├── overrides.go
│   │   ├── auth_middleware.go           # shared admin/admin
session check
│   │   └── *_test.go
│   ├── logging/                         # NEW — credential-
redacting log writer
│   │   ├── redactor.go
│   │   └── *_test.go
│   └── service/                         # existing
└── Dockerfile.jackett                   # NEW
```

All shared mutable state across goroutines uses `digital.vasic.concurrency/pkg/safe` (CONST-13). No bare `sync.Mutex + map` patterns.

5. SQLite Schema (`config/boba.db`)

Migration `001_init.sql`:

> **Note:** PRAGMA settings (`journal_mode=WAL`, `foreign_keys=ON`, `synchronous=NORMAL`) live in the connection DSN in `internal/db/conn.go`, not in this migration file. SQLite rejects `PRAGMA synchronous` inside a transaction, and the migrations stepper wraps each migration in a transaction for atomic apply. Setting them per-connection is the only place they actually take effect for `journal_mode` and `foreign_keys` anyway.

```
CREATE TABLE credentials (
  name              TEXT PRIMARY KEY,
  kind              TEXT NOT NULL CHECK (kind IN ('userpass',
        'cookie')),
  username_enc      BLOB,
  password_enc      BLOB,
  cookies_enc       BLOB,
  created_at        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  last_used_at      DATETIME
);

CREATE TABLE indexers (
  id                        TEXT PRIMARY KEY,
  display_name              TEXT NOT NULL,
  type                      TEXT NOT NULL,
  configured_at_jackett     INTEGER NOT NULL,
  linked_credential_name    TEXT REFERENCES credentials(name) ON
        DELETE SET NULL,
  enabled_for_search        INTEGER NOT NULL DEFAULT 1,
  last_jackett_sync_at      DATETIME,
  last_test_status          TEXT,
  last_test_at              DATETIME
);
CREATE INDEX idx_indexers_linked_cred ON
         indexers(linked_credential_name);

CREATE TABLE catalog_cache (
  id                    TEXT PRIMARY KEY,
  display_name          TEXT NOT NULL,
  type                  TEXT NOT NULL,
  language              TEXT,
  description           TEXT,
  template_fields_json  TEXT NOT NULL,
  cached_at             DATETIME NOT NULL
);
CREATE INDEX idx_catalog_cached_at ON catalog_cache(cached_at);

CREATE TABLE autoconfig_runs (
  id                        INTEGER PRIMARY KEY AUTOINCREMENT,
  ran_at                    DATETIME NOT NULL,
  discovered_count          INTEGER NOT NULL,
  configured_now_count      INTEGER NOT NULL,
  errors_json               TEXT NOT NULL DEFAULT '[]',
```

```
  result_summary_json      TEXT NOT NULL
);
CREATE INDEX idx_runs_ran_at ON autoconfig_runs(ran_at DESC);

CREATE TABLE indexer_map_overrides (
  env_name      TEXT PRIMARY KEY,
  indexer_id    TEXT NOT NULL,
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- IF NOT EXISTS + INSERT OR IGNORE because Migrate() bootstraps
       this
-- same table first so it can query for applied versions before
       running
-- the migration body. Idempotency keeps fresh and resumed boots
       safe.
CREATE TABLE IF NOT EXISTS schema_migrations (
  version       INTEGER PRIMARY KEY,
  applied_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT OR IGNORE INTO schema_migrations (version) VALUES (1);
```

5.1 Encryption envelope

`*_enc` columns store `nonce(12B) || ciphertext_with_GCM_tag`. Helper `internal/db/crypto.go`:

```
func Encrypt(key []byte, plaintext string) ([]byte, error)  //
       returns nonce||ct
func Decrypt(key []byte, blob []byte) (string, error)
```

Empty values are stored as `NULL`, never as encrypted-empty (avoids leaking presence vs. absence).

5.2 File permissions

Strictly enforced at startup; failure aborts boot:

- `config/boba.db` and `config/boba.db-wal`, `config/boba.db-shm` → `0600`
- `.env` → `0600`
- A challenge script verifies these in the running container.

6. Bootstrap & Migration Flow

On every `boba-jackett` start:

1. **Load `.env`** from `BOBA_ENV_PATH`. If file missing, abort with clear error.

2. **Ensure `BOBA_MASTER_KEY`**:
    - If absent or empty in `.env`, generate 32 bytes via `crypto/rand`, hex-encode (64 chars).
    - Append to `.env` atomically with this header block:

      ```
      # === BOBA SYSTEM ===
      # Master key for credential encryption-at-rest in config/boba.db.
      # DO NOT LOSE THIS — credentials become unrecoverable without it.
      # To rotate: see docs/BOBA_DATABASE.md § "Key Rotation".
      BOBA_MASTER_KEY=<hex>
      ```

    - Re-read `.env`. Log `"BOBA_MASTER_KEY auto-generated and persisted to .env"`.
3. **Open DB** at `BOBA_DB_PATH`. If file missing, create + apply all migrations in `internal/db/migrations/`.
4. **Run any pending migrations** (compare `schema_migrations.version` to embedded migrations). Each migration runs in its own transaction.
5. **First-run import** (only when `credentials` table is empty):
    - Scan loaded env for `<NAME>_USERNAME/_PASSWORD/_COOKIES` triples (same regex as autoconfig: `^([A-Z][A-Z0-9_]+?)_(USERNAME|PASSWORD|COOKIES)$`).
    - Apply the same denylist as the existing autoconfig (`QBITTORRENT,JACKETT,WEBUI,PROXY,MERGE,BRIDGE`).
    - For each complete bundle, insert encrypted row.
    - Log `"imported N credentials from .env"`.
6. **Idempotent re-import** (`credentials` table non-empty):
    - Insert ONLY env triples whose `name` is not already a row. **Existing rows are not overwritten** — DB wins.
    - Log `"imported N new credentials from .env, M already in DB"`.
7. **Apply credential map overrides**: load `indexer_map_overrides` rows; merge with `JACKETT_INDEXER_MAP` env var (DB wins).
8. **Run autoconfig once**: same `autoconfigure_jackett` semantics as the Python impl, but reading creds from DB and overrides from DB. Result row inserted into `autoconfig_runs`.
9. Start HTTP server on `PORT`.

7. Sync Semantics

Trigger	DB write	`.env` write	Jackett write
User adds/edits cred via dashboard	encrypted upsert	atomic mirror (tmp+fsync+rename)	replay autoconfig for that

Trigger	DB write	`.env` write	Jackett write
			single tracker
User removes cred via dashboard	row delete	remove `<NAME>_*` triple	`DELETE /api/v2.0/indexers/{id}`
User edits `.env` manually + restart	insert if not exists (step 6)	(already done)	autoconfig on boot
User adds indexer via Jackett UI directly (`:9117`)	upsert `indexers` row on next sync poll	no env write	(already done by user)
User toggles `enabled_for_search`	update row		(search filter applied at query time)
Catalog refresh (TTL `24h` or manual button)	`catalog_cache` upsert		(read-only fetch)
`BOBA_MASTER_KEY` regenerated (manual rotation)	re-encrypt all rows in transaction	overwrite key	

7.1 Atomic `.env` writes

Implemented in `internal/envfile/write.go`:

1. Read entire current `.env` into memory.
2. Apply requested mutation (insert/update/delete one or more triples).
3. Write to `<envpath>.tmp` with mode `0600`.
4. `fsync(.tmp)`.
5. `rename(.tmp, <envpath>)` (atomic on POSIX same-FS).
6. `fsync(parent_dir)`.

A single `sync.Mutex` in the writer guards against concurrent dashboard requests.

7.2 Conflict policy

Dashboard writes always win. If a user manually edits `.env` while the service is running and then triggers a restart, the manual change is imported only if the row isn't already in the DB — DB is the running source of truth.

For users who NEED a manual `.env` edit to take effect on a tracker that already has a DB row: documented procedure is "delete the row via dashboard, then restart" (or `POST /api/v1/jackett/credentials/{name}` overwrite via API).

8. API Surface (Go service on `:7189`)

All endpoints under `/api/v1/jackett/`. All `POST/PATCH/DELETE` require the existing admin/admin session cookie (validated via shared middleware `internal/jackettapi/auth_middleware.go`).

8.1 Credentials

```
GET    /credentials
       → 200 [{name, kind, has_username, has_password,
has_cookies,
               created_at, updated_at, last_used_at}]
       Never returns plaintext values.

POST   /credentials
       Body: {name, username?, password?, cookies?}
       PATCH semantics — only fields present are updated.
       → 200 {name, kind, has_username, ...}

DELETE /credentials/{name}
       Cascade: linked_credential_name → NULL on indexers; ON
DELETE SET NULL.
       → 204
```

8.2 Indexers

```
GET    /indexers
       → 200 [{id, display_name, type, configured_at_jackett,
               linked_credential_name, enabled_for_search,
               last_jackett_sync_at, last_test_status,
last_test_at}]

POST   /indexers/{id}
       Body: {credential_name, extra_fields?}
       Configures (or reconfigures) the indexer in Jackett.
Upserts row.
       → 200 indexer object
```

```
DELETE /indexers/{id}
       Removes from Jackett + deletes row.
       → 204

POST   /indexers/{id}/test
       Triggers a test query against the indexer.
       → 200 {status:
"ok"|"auth_failed"|"unreachable"|"empty_results", details?}
       Side-effect: updates last_test_status + last_test_at.

PATCH  /indexers/{id}
       Body: {enabled_for_search: bool}
       → 200 indexer object
```

8.3 Catalog

```
GET    /catalog?search=&type=&language=&page=&page_size=
       Paginated query over catalog_cache. page_size defaults 50,
max 200.
       → 200 {total, page, page_size, items: [{id, display_name,
type,
                                               language,
description,
                                               required_fields:
[...]}]}

POST   /catalog/refresh
       Force re-fetch from Jackett. Returns when complete
(typically <2s).
       → 200 {refreshed_count, errors: []}
```

8.4 Autoconfig runs

```
GET    /autoconfig/runs?limit=50
       → 200 [{id, ran_at, discovered_count, configured_now_count,
               error_count}]
       Paginated by limit only (no offset; UI shows last N).

GET    /autoconfig/runs/{id}
       → 200 full redacted result_summary_json

POST   /autoconfig/run
       Triggers a synchronous autoconfig run.
       → 200 full redacted run summary
```

8.5 Indexer-name overrides

```
GET    /overrides
       → 200 [{env_name, indexer_id, created_at}]
```

```
POST   /overrides
       Body: {env_name, indexer_id}
       Upsert.
       → 200 row

DELETE /overrides/{env_name}
       → 204
```

8.6 Health

```
GET    /healthz
       → 200 {status, db_ok, jackett_ok, version, uptime_s}
```

8.7 OpenAPI

`GET /openapi.json` exposes a generated OpenAPI 3.1 spec; contract tests (Layer 6) validate live responses against it.

9. Dashboard UI (Angular)

Two new top-level routes under `/jackett`. Implementation uses Angular Material to match the existing dashboard.

9.1 `/jackett/credentials`

Table view — columns: name, kind badge, presence badges (has-user / has-pass / has-cookies), last-used-at, actions (edit / delete).

“Add credential” button → modal with fields: - Tracker name (uppercase, validated against `^[A-Z][A-Z0-9_]+$`) - Kind selector (userpass / cookie) - Conditional fields based on kind

Edit modal — masked input fields. **Empty field = unchanged** (PATCH semantics, mirrored from API). User may explicitly clear a field via a “Clear” button next to it.

Confirmation modal on delete shows linked indexers that will lose their credential link.

9.2 `/jackett/indexers`

Three tabs:

Tab 1: Configured

Table of currently configured indexers (`indexers.configured_at_jackett = 1`). Columns: status badge (green=ok, yellow=stale, red=auth_failed/unreachable), id, display name, linked credential, enabled-for-search toggle, last test, actions (test / remove / edit).

Tab 2: Browse Catalog

Searchable table over catalog cache. Filters: text search (name + id + description), type (public/private/semi-private), language. Paginated 50/page.

Each row has an "Add" button. Clicking opens a configuration modal pre-populated with: - The indexer's required fields (from `template_fields_json`) - A credential picker — auto-suggests a credential whose name fuzzy-matches the indexer (the same matcher as autoconfig), or "Create new credential" inline - A "Save" button that configures via Jackett + writes the row

Tab 3: Autoconfig History

Last 50 run entries (most recent first). Click row → expands to show the full redacted `result_summary_json` with syntax-highlighted JSON viewer. "Run autoconfig now" button at the top.

9.3 IPTorrents cookie capture flow

When the user clicks "Add" on `iptorrents` (or any indexer whose required-fields include `cookie` / `cookies` / `cookieheader`), the modal opens with a step-by-step instruction panel:

> 1. Open `https://iptorrents.com` in a new browser tab and log in.
> 2. Open DevTools (F12) → Application → Storage → Cookies → `iptorrents.com`.
> 3. Copy the values of the `uid` and `pass` cookies.
> 4. Format as `uid=<value>; pass=<value>` and paste below.

Single textarea for the cookie blob, "Save" writes encrypted cookie row + triggers configure for that indexer.

9.4 NNMClub clarification

A small info banner on `/jackett/credentials` for any tracker name that exists in the credentials table but has no Jackett indexer match (after catalog has been loaded once):

> NNMCLUB credentials present but no Jackett indexer matches. NNMClub is served by the native nova3 plugin (`plugins/nnmclub.py`) — these credentials are still used. No further action required.

The autoconfig result `skipped_no_match` field gets a parallel `served_by_native_plugin` field for any name that matches a known native-plugin filename in `plugins/`.

9.5 Routing + nav

Top nav gains a single “Jackett” entry. Click → `/jackett` (default child route `/jackett/credentials`).

10. Test Strategy

Resource-limited per CONST-09:
`GOMAXPROCS=2 nice -n 19 ionice -c 3` for all `go test` invocations.
Container memory limits already in compose.

10.1 Layer 1 — Unit (Go `_test.go`, `-short`, mocks OK)

Module	Tests
`internal/db/crypto`	round-trip; nonce uniqueness over 100k samples; tamper detection (flip 1 bit → fail); empty plaintext rejected
`internal/db/migrate`	apply 0→latest; idempotency; transaction rollback on bad SQL; embedded files match disk
`internal/envfile`	parse all forms (quoted/unquoted/comments/blanks); atomic-write under crash sim (kill -9 between tmp + rename); whitespace + duplicate-key handling
`internal/jackett/matcher`	port of Python tests verbatim — fuzzy threshold, override precedence, ambiguous detection
`internal/jackett/autoconfig`	port of Python tests — denylist, complete/incomplete bundles, total-timeout, error envelope
`internal/logging/redactor`	

Module	Tests
	every credential value present in the database is replaced by *** in any log line

10.2 Layer 2 — Integration (real SQLite + real `.env`, no Jackett)

`tests/integration/jackett_db_test.go`:

- Bootstrap from empty `.env` → master key generated, written, file mode 0600.
- Bootstrap from `.env` with N triples → all imported, encrypted, table count = N.
- Restart with same `.env` → no re-imports, log shows "0 new, N already in DB".
- UI add cred → DB row + `.env` line both present and consistent.
- UI delete cred → DB row gone + `.env` line gone.
- Concurrent dashboard writes via 50 goroutines → no `.env` corruption (line count + parse check after each).

10.3 Layer 3 — E2E (real Go service + real Jackett container + real `.env`)

`tests/e2e/jackett_management_test.go`:

- Boot full stack; add `RUTRACKER` cred via API → autoconfig runs → indexer appears in Jackett's `/api/v2.0/indexers`.
- Browse catalog → 620 entries returned.
- Add `iptorrents` with cookie via API → indexer configured in Jackett with cookie value.
- Test indexer → status="ok" returned.
- Toggle `enabled_for_search` → search request to merge service excludes that indexer.
- Delete cred → indexer gone from Jackett.
- Pull `JACKETT_INDEXER_MAP` from env → DB override row created → autoconfig uses it.

10.4 Layer 4 — Security / Penetration (THE leak-prevention requirement)

`tests/security/credential_leak_test.go` + `challenges/scripts/credential_leak_grep_challenge.sh`:

- Insert 6 unique high-entropy credentials (so any leak is greppable with low false-positive rate).
- Drive a full E2E flow exercising every endpoint in §8.

- Capture: all log output (stdout + stderr + file), every HTTP response body across the flow, full `boba.db` hex dump, full process `/proc/<pid>/environ`.
- Grep all captures for the literal credential values.
- **Assertion**: 0 hits in logs, 0 hits in HTTP responses, 0 hits in `/proc/<pid>/environ` (env vars containing the values must not be set on the process — only `BOBA_MASTER_KEY` should be).
- Hits in `boba.db` ARE allowed but must be encrypted (verify by attempting to decrypt with the wrong key → fail; with the right key → match).

Static scans: - `gosec ./...` clean - `bandit -r download-proxy/src/` clean

File permissions: - `config/boba.db`, `config/boba.db-wal`, `config/boba.db-shm` mode `0600` - `.env` mode `0600` - A challenge script asserts these in the running container.

10.5 Layer 5 — Benchmark

`internal/db/repos/*_bench_test.go`:

- `BenchmarkCatalogQuery` — over the 620-row cache with these scenarios: (a) no filter (full page), (b) text search 3-char prefix, (c) `type=private` filter alone, (d) text search + language filter combined; each scenario p99 < 50ms on host hardware.
- `BenchmarkEncryptDecrypt` — p99 < 1ms per op.
- `BenchmarkAutoconfigFullRun` — full autoconfig with 5 mock indexers; p99 < 5s.

10.6 Layer 6 — Contract

OpenAPI 3.1 generated at build time. `tests/contract/openapi_test.go` hits every endpoint and validates the response shape against the spec.

10.7 Layer 7 — Challenge (CONST-02 + CONST-32 reproduction-before-fix)

In `challenges/scripts/`:

1. `cred_roundtrip_challenge.sh` — POST cred → GET should return metadata only → DB row encrypted → `.env` line plaintext.
2. `env_db_drift_challenge.sh` — make 100 changes via API, assert `.env` and DB are byte-equivalent in semantics after each.
3. `iptorrents_cookie_flow_challenge.sh` — follow the documented flow → indexer configured with cookie field populated.
4. `master_key_autogen_challenge.sh` — fresh `.env` without `BOBA_MASTER_KEY` → service starts → key appears in `.env` with the

documented header → restart → key is unchanged (idempotent).
5. `credential_leak_grep_challenge.sh` — wrapper around the Layer 4 test for inclusion in `run_all_challenges.sh`.
6. `nnmclub_native_plugin_clarification_challenge.sh` — autoconfig output for NNMCLUB shows `served_by_native_plugin` flag.
7. `boba_db_file_perms_challenge.sh` — assert `0600` on all DB and env files.

All registered in `challenges/scripts/run_all_challenges.sh`.

11. Definition of Done (the "no loose ends" requirement)

A single checklist that MUST be before declaring this work complete:

1. **All endpoints in §8 implemented** + Layer 2 integration tested + Layer 3 E2E tested against live stack.
2. **Both UI pages implemented** + manually verified in browser via Playwright walkthroughs (golden path + 5 edge cases per page documented in implementation plan).
3. **All 7 test layers green**:
   - Layer 1 (unit) `go test -short -race ./...` — pass
   - Layer 2 (integration) — pass
   - Layer 3 (E2E) — pass
   - Layer 4 (security) — leak grep 0 hits; gosec/bandit clean; file perms enforced
   - Layer 5 (bench) — all p99 thresholds met
   - Layer 6 (contract) — OpenAPI matches every response
   - Layer 7 (challenges) — all 7 challenge scripts pass via `run_all_challenges.sh`
4. **Plan doc reconciled**: `docs/superpowers/plans/2026-04-26-jackett-autoconfig-clean-rebuild.md` checkboxes updated to reflect what shipped (currently 84 unchecked, 0 checked even though Layers 1-7 are in commits).
5. **Documentation updated**:
   - `CLAUDE.md` § "Architecture" — adds `boba-jackett` service + ports map
   - `AGENTS.md` — same updates
   - `docs/JACKETT_INTEGRATION.md` — auto-config section rewritten as DB-backed
   - `docs/BOBA_DATABASE.md` — NEW: schema reference, master key lifecycle, key rotation procedure, backup procedure, recovery from lost key
6. **Bugfix doc**: every bug surfaced + fixed during implementation logged in `docs/issues/fixed/BUGFIXES.md` per CONST-10.
7. **`IPTORRENTS_COOKIES`** documented in `docs/JACKETT_INTEGRATION.md` § "Cookie-only indexers" and surfaced in the IPTorrents add modal.

8. **NNMClub clarification** in autoconfig output AND in dashboard banner.
9. **Pre-test commit + push to all upstreams** — every change committed via Conventional Commits and pushed to **every configured remote of the main repo** (currently `origin`, `github`, `upstream` — all aliased to `git@github.com:milos85vasic/qBitTorrent.git`, but each must receive its own `git push <remote> <branch>` invocation) **before** containers are booted for the final test/challenge run. If submodules are added in the future (none today — verified `.gitmodules` absent on 2026-04-27), each must also be committed + pushed to all its remotes before parent-repo push.
10. **Final demo block** in PR body with pasted output from the real end-to-end run (CONST Definition of Done).
11. **Open-issues sweep**: `grep -rE 'TODO|FIXME|XXX|HACK' qBitTorrent-go/ download-proxy/src/ frontend/src/ docs/` produces zero new entries from this work. Pre-existing entries are left alone unless touched.
12. **CONST-013 audit**: no bare `sync.Mutex + map/slice` in any new Go code; all mutable shared collections use `safe.Store` / `safe.Slice`.
13. **CONST-033 verification**: `bash challenges/scripts/no_suspend_calls_challenge.sh` AND `bash challenges/scripts/host_no_auto_suspend_challenge.sh` both pass after all changes are committed.

12. Risks & Mitigations

Risk	Mitigation
Master key lost → all credentials unrecoverable	(a) Auto-generated key written to `.env` with prominent header; (b) `docs/BOBA_DATABASE.md` documents backup procedure; (c) on key-mismatch detection at startup, abort with explicit "key mismatch — restore your `.env` from backup, or wipe `boba.db` to re-import from `.env`" error
`.env` corruption during atomic write	tmp + fsync + rename + parent-dir fsync; concurrent-writer mutex; integration test simulates kill -9 mid-write
Two backends drift on shared concepts (autoconfig output shape)	OpenAPI spec is authoritative; Python `/api/v1/jackett/autoconfig/last` endpoint is REMOVED in this work — single owner

Risk	Mitigation
Browse-catalog page slow with 620 rows	Server-side pagination + index on `cached_at`; benchmark gate at p99 < 50ms
Manual `.env` edits silently ignored after first import	Documented in `docs/JACKETT_INTEGRATION.md`; future dashboard banner on detected drift
Re-encryption migration breaks if interrupted	Run inside a single SQLite transaction; on failure roll back, keep old key

13. Out of Scope (Phase 2+)

- Port merge search to Go.
- Multi-user auth / per-user credential isolation.
- Postgres backend.
- Master key rotation UI (manual procedure documented; scripted in Phase 2).
- OAuth-based credential acquisition for trackers that support it.
- Backup/restore UI (manual `cp` documented).

14. References

- `docs/JACKETT_INTEGRATION.md` — current Jackett integration documentation
- `docs/superpowers/specs/2026-04-26-jackett-autoconfig-clean-rebuild-design.md` — predecessor spec for the autoconfig itself
- `docs/superpowers/plans/2026-04-26-jackett-autoconfig-clean-rebuild.md` — implementation plan for the autoconfig (checkboxes need reconciliation per §11.4)
- `download-proxy/src/merge_service/jackett_autoconfig.py` — reference Python implementation to port
- `CLAUDE.md` § "Universal Mandatory Constraints" — CONST-01, CONST-02, CONST-09, CONST-10, CONST-11, CONST-13, CONST-32, CONST-33
- `CONSTITUTION.md` § CONST-033 — host power management ban